Building a Global Datacenter Dataset: Malaysia Pilot (v2)

Daniel

2026-04-16

Table of contents

1 Project Overview **2**

2 Part 1: Environment Setup **2**
2.1 Configure reticulate and load R packages . . . 2
2.2 Verify Python environment . . . 3
2.3 Stage the Natural Earth shapefile . . . 3

3 Part 2: Query OpenStreetMap via Overpass **4**
3.1 Build the Overpass query . . . 4
3.2 Send the request . . . 5
3.3 Parse the response into a tibble . . . 6

4 Part 3: Query Wikidata via SPARQL **7**
4.1 Build the SPARQL query . . . 7
4.2 Send the SPARQL request . . . 8
4.3 Parse the SPARQL response . . . 9

5 Part 4: Cloud Provider Hyperscale Regions **10**
5.1 Query AWS regional services JSON . . . 10
5.2 Compile Azure, GCP, Oracle, and Alibaba regions . . . 12
5.3 Combine cloud sources into a single tibble . . . 13

6 Part 5: Merge and Deduplicate **14**
6.1 Union the three sources . . . 14
6.2 Deduplicate by spatial proximity . . . 15

7 Part 6: Spatial Enrichment with Natural Earth **17**
7.1 Load country boundaries and join . . . 17

8 Part 7: Python Enrichment via reticulate **18**
8.1 Switch to Python for text normalization . . . 18
8.2 Pull the Python dataframe back into R . . . 20

9 Part 8: Visualization **21**
9.1 Interactive leaflet map 21
9.2 Source coverage summary 22

10 Part 9: Export **24**
10.1 Write the final dataset 24

11 Reflection **24**

1 Project Overview

I wanted to build a replicable pipeline for constructing a global datacenter dataset from openly-licensed sources. Commercial directories like Data Center Map prohibit scraping and manual extraction in their terms of service, so I built this around three open sources instead: OpenStreetMap via the Overpass API, Wikidata via SPARQL, and cloud provider region endpoints. This document implements the pipeline for a single country — Malaysia — as a pilot. Malaysia was chosen deliberately: Johor has become one of the most active hyperscale build-out markets in Southeast Asia since 2023, which means the country is small enough to run end-to-end in seconds but large enough to produce an interesting dataset with real analytical value.

Version 2 of this pipeline adds cloud provider regions as a third source layer, following a finding from v1 that OSM and Wikidata together undercount hyperscale infrastructure. AWS, Azure, GCP, Oracle, and Alibaba have all announced or activated Malaysia regions since 2022, but none appear in OSM or Wikidata as named facilities. Closing that gap was the motivation for v2.

The deliverable is a tidy dataset of datacenter facilities in Malaysia with names, operators where available, coordinates, source attribution, and country-level metadata joined from Natural Earth shapefiles. The pipeline mixes R and Python via reticulate, following the same pattern I used in Labs 7 and 8.

2 Part 1: Environment Setup

2.1 Configure reticulate and load R packages

I began by pointing reticulate at the `textmining` conda environment before loading any packages that touch Python. Setting `RETICULATE_PYTHON` as an environment variable belt-and-suspenders alongside the YAML `python:` field is the pattern that worked reliably in Lab 8, especially when spacyr is in the mix. I then loaded the R packages needed for HTTP, JSON parsing, spatial operations, and mapping.

```
Sys.setenv(RETICULATE_PYTHON = "/opt/anaconda3/envs/textmining/bin/python")

library(reticulate)
use_condaenv("textmining", required = TRUE)

library(httr)
```

```r
library(jsonlite)
library(dplyr)
library(tidyr)
library(stringr)
library(purrr)
library(sf)
library(leaflet)
library(ggplot2)
library(knitr)
```

2.2 Verify Python environment

I confirmed reticulate was using the correct Python binary before moving on. This is the check that caught environment drift problems in Lab 10.

```r
py_config()
```

```
python:         /opt/anaconda3/envs/textmining/bin/python
libpython:      /opt/anaconda3/envs/textmining/lib/libpython3.13.dylib
pythonhome:     /opt/anaconda3/envs/textmining:/opt/anaconda3/envs/textmining
version:        3.13.11 | packaged by Anaconda, Inc. | (main, Dec 10 2025, 21:21:08) [Clang 20
numpy:          /opt/anaconda3/envs/textmining/lib/python3.13/site-packages/numpy
numpy_version:  2.4.1

NOTE: Python version was forced by RETICULATE_PYTHON
```

2.3 Stage the Natural Earth shapefile

Before running the pipeline I made sure the Natural Earth country shapefile was available locally. In v1 I had to chase this file across project directories, so this chunk copies it from the Lab 8 project if it's not already staged in this project's `data/` folder. If it can't find the shapefile anywhere, the spatial join step in Part 5 has a graceful fallback that hand-codes Malaysia metadata.

```r
if (!dir.exists("data")) dir.create("data")
if (!dir.exists("data/world-shape-files")) dir.create("data/world-shape-files")

dst_dir <- "data/world-shape-files"
ne_staged <- list.files(dst_dir, pattern = "^ne_50m_admin_0_countries\\.shp$")

if (length(ne_staged) == 0) {
  src_candidates <- c(
    "lab-8-collaborative-data-visualization-psg2/data/world-shape-files",
    "../lab-8-collaborative-data-visualization-psg2/data/world-shape-files"
  )
  for (src_dir in src_candidates) {
```

```r
    if (dir.exists(src_dir)) {
      files_to_copy <- list.files(src_dir,
                                  pattern = "^ne_50m_admin_0_countries",
                                  full.names = TRUE)
      if (length(files_to_copy) > 0) {
        file.copy(files_to_copy, dst_dir, overwrite = TRUE)
        break
      }
    }
  }
}
```

3 Part 2: Query OpenStreetMap via Overpass

3.1 Build the Overpass query

OpenStreetMap contributors tag datacenters using two main schemas: `telecom=data_center` (the canonical tag) and `building=data_center` (used when the tag is attached to the building geometry rather than an amenity node). I queried for both, scoped to Malaysia using the ISO country code `MY`. The `[out:json]` header returns JSON, `out center tags` asks Overpass to return the centroid coordinates for ways and relations rather than their full geometry, which keeps the payload small.

```r
overpass_query <- paste(
  '[out:json][timeout:180];',
  'area["ISO3166-1"="MY"]->.my;',
  '(',
  '  node["telecom"="data_center"](area.my);',
  '  way["telecom"="data_center"](area.my);',
  '  relation["telecom"="data_center"](area.my);',
  '  node["building"="data_center"](area.my);',
  '  way["building"="data_center"](area.my);',
  '  relation["building"="data_center"](area.my);',
  ');',
  'out center tags;',
  sep = "\n"
)

cat(overpass_query)
```

```
[out:json][timeout:180];
area["ISO3166-1"="MY"]->.my;
(
  node["telecom"="data_center"](area.my);
  way["telecom"="data_center"](area.my);
  relation["telecom"="data_center"](area.my);
```

```
  node["building"="data_center"](area.my);
  way["building"="data_center"](area.my);
  relation["building"="data_center"](area.my);
);
out center tags;
```

3.2 Send the request

I sent the query to the main Overpass endpoint. The response comes back as JSON with an `elements` array where each element is a node, way, or relation with its tags and coordinates. I added a user-agent header as a courtesy — Overpass operators appreciate knowing who is hitting their endpoint, and it reduces the chance of being throttled. The chunk uses a cached response if one exists from a previous run, which saves network time during iteration and also makes the pipeline resilient to transient Overpass outages.

```r
cache_file <- "outputs/overpass_malaysia_raw.json"

if (!dir.exists("outputs")) dir.create("outputs")

if (file.exists(cache_file)) {
  raw <- readLines(cache_file) |> paste(collapse = "\n")
  cat("Loaded cached Overpass response from", cache_file, "\n")
} else {
  overpass_resp <- POST(
    "https://overpass-api.de/api/interpreter",
    body = list(data = overpass_query),
    encode = "form",
    add_headers(`User-Agent` = "ITEC724-DatacenterPilot/1.0"),
    timeout(200)
  )

  cat("Status:", status_code(overpass_resp), "\n")
  raw <- content(overpass_resp, as = "text", encoding = "UTF-8")

  if (startsWith(trimws(raw), "<")) {
    cat("Overpass returned an error page. First 1000 chars:\n")
    cat(substr(raw, 1, 1000))
    stop("Overpass returned HTML/XML instead of JSON")
  }

  writeLines(raw, cache_file)
  cat("Saved fresh response to", cache_file, "\n")
}
```

```
Loaded cached Overpass response from outputs/overpass_malaysia_raw.json
```

```r
cat("First 300 chars of response:\n")
```

```
First 300 chars of response:
```

```r
cat(substr(raw, 1, 300))
```

```
{
  "version": 0.6,
  "generator": "Overpass API 0.7.62.11 87bfad18",
  "osm3s": {
    "timestamp_osm_base": "2026-04-16T15:25:30Z",
    "timestamp_areas_base": "2026-04-16T06:15:20Z",
    "copyright": "The data included in this document is from www.openstreetmap.org. The data is
```

3.3 Parse the response into a tibble

The raw JSON needed flattening. Nodes have `lat`/`lon` directly; ways and relations have their centroid nested under `center`. I unified both into a single tibble with consistent columns and pulled the most useful tags out of the nested `tags` object — name, operator, and the two datacenter-identifying tags. Any row missing both a name and an operator got a synthetic label so downstream joins still work.

```r
parsed <- fromJSON(raw, flatten = TRUE)

elements <- parsed$elements

get_col <- function(df, colname) {
  if (colname %in% names(df)) df[[colname]] else rep(NA, nrow(df))
}

osm_df <- tibble(
  osm_id   = elements$id,
  osm_type = elements$type,
  lat      = ifelse(is.na(get_col(elements, "lat")),
                    get_col(elements, "center.lat"),
                    get_col(elements, "lat")),
  lon      = ifelse(is.na(get_col(elements, "lon")),
                    get_col(elements, "center.lon"),
                    get_col(elements, "lon")),
  name     = get_col(elements, "tags.name"),
  operator = get_col(elements, "tags.operator"),
  telecom  = get_col(elements, "tags.telecom"),
  building = get_col(elements, "tags.building")
) %>%
  mutate(
```

```
    name = ifelse(is.na(name) | name == "",
                  paste0("Unnamed facility (OSM ", osm_type, " ", osm_id, ")"),
                  name),
    source = "OpenStreetMap"
  ) %>%
  filter(!is.na(lat), !is.na(lon))

cat("Parsed", nrow(osm_df), "datacenter facilities from OSM\n\n")
```

```
Parsed 20 datacenter facilities from OSM
```

```
kable(head(osm_df, 10), caption = "First 10 datacenters from OpenStreetMap")
```

Table 1: First 10 datacenters from OpenStreetMap

osm_id	osm_type	lat	lon	name	operator	telecom	building	source
7001771562	node	5.405762	103.0913	Server Room B2	NA	data_center	NA	OpenStreetMap
10939660647	node	3.174386	101.7294	MAF War Gaming Simulation Center	NA	data_center	NA	OpenStreetMap
13197364001	node	1.709689	110.3534	IRIX DC	NA	data_center	NA	OpenStreetMap
848498975	way	2.932535	101.6573	TMONE Klang Valley Data Center	NA	data_center	NA	OpenStreetMap
1080495340	way	2.937752	101.6574	AIMS	NA	data_center	NA	OpenStreetMap
1432337885	way	1.703383	103.4134	Unnamed facility (OSM way 1432337885)	NA	data_center	data_center	OpenStreetMap
1432337898	way	1.693983	103.4190	Unnamed facility (OSM way 1432337898)	NA	data_center	data_center	OpenStreetMap
1432337899	way	1.692135	103.4124	Unnamed facility (OSM way 1432337899)	NA	data_center	data_center	OpenStreetMap
1432337900	way	1.693099	103.4179	Unnamed facility (OSM way 1432337900)	NA	data_center	data_center	OpenStreetMap
1432337901	way	1.692396	103.4171	Unnamed facility (OSM way 1432337901)	NA	data_center	data_center	OpenStreetMap

4 Part 3: Query Wikidata via SPARQL

4.1 Build the SPARQL query

Wikidata's structured data sits behind a SPARQL endpoint. I queried for any item that is an `instance of` data center (Q1149652) or a subclass thereof, located in Malaysia (Q833), and with

geographic coordinates. The `SERVICE wikibase:label` block asks Wikidata to return human-readable labels in English, which saves a separate lookup for each item.

```
sparql_query <- '
SELECT ?item ?itemLabel ?coords ?operatorLabel ?inceptionDate WHERE {
  ?item wdt:P31/wdt:P279* wd:Q1149652 .
  ?item wdt:P17 wd:Q833 .
  ?item wdt:P625 ?coords .
  OPTIONAL { ?item wdt:P137 ?operator . }
  OPTIONAL { ?item wdt:P571 ?inceptionDate . }
  SERVICE wikibase:label { bd:serviceParam wikibase:language "en" . }
}
'

cat(sparql_query)
```

```
SELECT ?item ?itemLabel ?coords ?operatorLabel ?inceptionDate WHERE {
  ?item wdt:P31/wdt:P279* wd:Q1149652 .
  ?item wdt:P17 wd:Q833 .
  ?item wdt:P625 ?coords .
  OPTIONAL { ?item wdt:P137 ?operator . }
  OPTIONAL { ?item wdt:P571 ?inceptionDate . }
  SERVICE wikibase:label { bd:serviceParam wikibase:language "en" . }
}
```

4.2 Send the SPARQL request

Wikidata's query service accepts GET requests with the query as a URL parameter and returns JSON when the `Accept` header is set appropriately.

```
wd_response <- GET(
  url = "https://query.wikidata.org/sparql",
  query = list(query = sparql_query, format = "json"),
  add_headers(
    `Accept`     = "application/sparql-results+json",
    `User-Agent` = "ITEC724-DatacenterPilot/1.0 (academic research)"
  )
)

status_code(wd_response)
```

```
[1] 200
```

4.3 Parse the SPARQL response

Wikidata returns coordinates as WKT-formatted strings like `Point(lon lat)`, which I parsed out with a regex. The result is a tidy tibble matching the OSM schema so the two sources can be unioned.

```
wd_raw <- content(wd_response, as = "text", encoding = "UTF-8")
wd_parsed <- fromJSON(wd_raw, flatten = TRUE)

bindings <- wd_parsed$results$bindings

safe_col <- function(df, colname) {
  if (is.data.frame(df) && colname %in% names(df)) df[[colname]] else NA_character_
}

if (!is.null(bindings) && length(bindings) > 0 && nrow(bindings) > 0) {

  wd_df <- tibble(
    wikidata_id = basename(safe_col(bindings, "item.value")),
    name        = safe_col(bindings, "itemLabel.value"),
    coords_wkt  = safe_col(bindings, "coords.value"),
    operator    = safe_col(bindings, "operatorLabel.value"),
    inception   = safe_col(bindings, "inceptionDate.value")
  ) %>%
    mutate(
      lon = as.numeric(str_extract(coords_wkt, "(?<=Point\\()-?[0-9\\.]+")),
      lat = as.numeric(str_extract(coords_wkt, "-?[0-9\\.]+(?=\\))")),
      source = "Wikidata"
    ) %>%
    select(-coords_wkt)

  cat("Retrieved", nrow(wd_df), "datacenters from Wikidata\n")

} else {

  wd_df <- tibble(
    wikidata_id = character(),
    name = character(),
    operator = character(),
    inception = character(),
    lon = numeric(),
    lat = numeric(),
    source = character()
  )

  cat("Wikidata returned no results for Malaysia\n")
}
```

```
Wikidata returned no results for Malaysia
```

```r
if (nrow(wd_df) > 0) {
  kable(wd_df, caption = "Datacenters from Wikidata")
} else {
  cat("(Empty table - dedup step will skip Wikidata source)\n")
}
```

```
(Empty table - dedup step will skip Wikidata source)
```

5 Part 4: Cloud Provider Hyperscale Regions

One clear limitation of the OSM and Wikidata sources was the absence of hyperscale cloud regions — the AWS, Azure, GCP, Oracle, and Alibaba datacenters that have been the main driver of Malaysian datacenter capacity growth since 2023. Hyperscalers rarely disclose facility-level addresses for security reasons, and OSM contributors generally can't tag buildings whose operator they can't confirm. To close this gap I added cloud provider region data as a third layer. I was honest in the approach: AWS publishes a machine-readable regional services JSON that I queried directly, but for Azure, GCP, Oracle, and Alibaba I used curated entries based on each provider's public announcements and regional documentation, with source attribution preserved. A future version of this pipeline should replace the curated lookup with per-provider API calls where available.

5.1 Query AWS regional services JSON

I started with AWS because it publishes the cleanest regional data of the five providers. The regional services JSON endpoint returns every service-region combination, which I filtered to the Malaysia region (`ap-southeast-5`, launched August 2024). Coordinates are approximated at central Kuala Lumpur since AWS does not publish facility-level locations.

```r
aws_regions_url <- "https://api.regional-table.region-services.aws.a2z.com/index.json"

aws_resp <- tryCatch(
  GET(
    aws_regions_url,
    add_headers(`User-Agent` = "ITEC724-DatacenterPilot/1.0"),
    timeout(30)
  ),
  error = function(e) { cat("AWS request error:", conditionMessage(e), "\n"); NULL }
)

if (!is.null(aws_resp) && status_code(aws_resp) == 200) {
  cat("AWS status:", status_code(aws_resp), "\n")
  aws_raw <- content(aws_resp, as = "text", encoding = "UTF-8")
  aws_parsed <- tryCatch(fromJSON(aws_raw, flatten = TRUE),
                         error = function(e) NULL)
```

```r
  if (!is.null(aws_parsed) && !is.null(aws_parsed$prices)) {
    aws_all <- aws_parsed$prices %>% as_tibble()

    region_col <- grep("region", names(aws_all), value = TRUE, ignore.case = TRUE)[1]

    if (!is.na(region_col)) {
      aws_my <- aws_all %>%
        filter(grepl("ap-southeast-5", .data[[region_col]],
                     ignore.case = TRUE)) %>%
        distinct(.data[[region_col]], .keep_all = TRUE) %>%
        transmute(
          name = "AWS Asia Pacific (Malaysia)",
          provider = "AWS",
          region_code = "ap-southeast-5",
          lat = 3.139, lon = 101.687,
          source = "Cloud Provider (AWS)",
          note = "Launched Aug 2024; physical location Selangor/KL area"
        )
    } else {
      aws_my <- tibble()
    }
  } else {
    aws_my <- tibble()
  }
} else {
  aws_my <- tibble()
}
```

```
AWS status: 200
```

```r
if (nrow(aws_my) == 0) {
  aws_my <- tibble(
    name = "AWS Asia Pacific (Malaysia)",
    provider = "AWS",
    region_code = "ap-southeast-5",
    lat = 3.139, lon = 101.687,
    source = "Cloud Provider (AWS)",
    note = "Launched Aug 2024; physical location Selangor/KL area"
  )
  cat("Using curated fallback for AWS Malaysia region\n")
}

kable(aws_my, caption = "AWS regions in Malaysia")
```

Table 2: AWS regions in Malaysia

name	provider	region_code	lat	lon	source	note
AWS Asia Pacific (Malaysia)	AWS	ap-southeast-5	3.139	101.687	Cloud Provider (AWS)	Launched Aug 2024; physical location Selangor/KL area

5.2 Compile Azure, GCP, Oracle, and Alibaba regions

For Azure, GCP, Oracle, and Alibaba, I assembled a curated tibble of their Malaysia regions based on each provider's publicly announced infrastructure pages. This is a known-limitation approach: machine-readable formats from these providers vary in structure and access requirements, so rather than pretend to scrape what would amount to hand-corrected data, I made the curation explicit in the source attribution. All four providers have active or announced Malaysia regions as of April 2026, concentrated in the Kuala Lumpur metro area.

```r
other_clouds <- tibble(
  name = c(
    "Azure Malaysia West",
    "Google Cloud asia-southeast3 (Malaysia)",
    "Oracle Cloud Kuala Lumpur (kix)",
    "Alibaba Cloud Malaysia (Kuala Lumpur)"
  ),
  provider = c("Azure", "GCP", "Oracle", "Alibaba"),
  region_code = c(
    "malaysiawest",
    "asia-southeast3",
    "ap-kuala-lumpur-1",
    "ap-southeast-3"
  ),
  lat = c(3.139, 3.140, 3.141, 3.142),
  lon = c(101.687, 101.688, 101.689, 101.690),
  source = c(
    "Cloud Provider (Azure)",
    "Cloud Provider (GCP)",
    "Cloud Provider (Oracle)",
    "Cloud Provider (Alibaba)"
  ),
  note = c(
    "Announced 2024, GA 2024; physical site Selangor/KL",
    "Announced 2024; physical site KL metro",
    "Active since 2022; KL metro",
    "Announced 2024; physical site KL metro"
  )
)
```

```r
kable(other_clouds,
      caption = "Azure, GCP, Oracle, Alibaba regions in Malaysia")
```

Table 3: Azure, GCP, Oracle, Alibaba regions in Malaysia

name	provider	region_code	lat	lon	source	note
Azure Malaysia West	Azure	malaysia-west	3.139	101.687	Cloud Provider (Azure)	Announced 2024, GA 2024; physical site Selangor/KL
Google Cloud asia-southeast3 (Malaysia)	GCP	asia-southeast3	3.140	101.688	Cloud Provider (GCP)	Announced 2024; physical site KL metro
Oracle Cloud Kuala Lumpur (kix)	Oracle	ap-kuala-lumpur-1	3.141	101.689	Cloud Provider (Oracle)	Active since 2022; KL metro
Alibaba Cloud Malaysia (Kuala Lumpur)	Alibaba	ap-southeast-3	3.142	101.690	Cloud Provider (Alibaba)	Announced 2024; physical site KL metro

I used slightly offset coordinates for the four curated providers (3.139 through 3.142 latitude, 101.687 through 101.690 longitude) rather than identical points. This is a deliberate choice: the dedup step in Part 5 rounds coordinates to three decimal places, which would collapse five facilities at identical coordinates into one. The 0.001-degree offsets are small enough to remain visually indistinguishable on the final map but large enough to survive dedup rounding, preserving each provider as a distinct row in the output. The underlying semantic claim stays honest: these are all approximately at Kuala Lumpur, with exact facility locations not publicly disclosed.

5.3 Combine cloud sources into a single tibble

I unified the five cloud entries into one tibble matching the schema used by OSM and Wikidata. This makes the Part 5 union step straightforward — all three sources become rows in the same combined dataset with source attribution preserved.

```r
cloud_df <- bind_rows(aws_my, other_clouds) %>%
  mutate(
    operator    = provider,
    wikidata_id = NA_character_,
    osm_id      = NA_character_,
    osm_type    = NA_character_
  ) %>%
  select(name, operator, lat, lon, source, osm_id, osm_type, wikidata_id,
         provider, region_code, note)

cat("Total cloud regions:", nrow(cloud_df), "\n")
```

```
Total cloud regions: 5
```

```r
kable(cloud_df %>% select(name, provider, region_code, source),
      caption = "All cloud regions combined")
```

Table 4: All cloud regions combined

name	provider	region_code	source
AWS Asia Pacific (Malaysia)	AWS	ap-southeast-5	Cloud Provider (AWS)
Azure Malaysia West	Azure	malaysiawest	Cloud Provider (Azure)
Google Cloud asia-southeast3 (Malaysia)	GCP	asia-southeast3	Cloud Provider (GCP)
Oracle Cloud Kuala Lumpur (kix)	Oracle	ap-kuala-lumpur-1	Cloud Provider (Oracle)
Alibaba Cloud Malaysia (Kuala Lumpur)	Alibaba	ap-southeast-3	Cloud Provider (Alibaba)

6 Part 5: Merge and Deduplicate

6.1 Union the three sources

I combined the OSM, Wikidata, and cloud tibbles into a single dataset, adding NA-padded columns where needed so all three sources share the same schema. OSM rows have `osm_id`/`osm_type`, Wikidata rows have `wikidata_id`, and cloud rows have `provider`/`region_code`/`note`. After the union, every row has every column — the source-specific columns just contain NA where not applicable.

```r
osm_slim <- osm_df %>%
  mutate(
    osm_id      = as.character(osm_id),
    osm_type    = as.character(osm_type),
    wikidata_id = NA_character_,
    provider    = NA_character_,
    region_code = NA_character_,
    note        = NA_character_
  ) %>%
  select(name, operator, lat, lon, source, osm_id, osm_type,
         wikidata_id, provider, region_code, note)

wd_slim <- wd_df %>%
  mutate(
    osm_id      = NA_character_,
    osm_type    = NA_character_,
    wikidata_id = as.character(wikidata_id),
```

```r
    provider    = NA_character_,
    region_code = NA_character_,
    note        = NA_character_
  ) %>%
  select(name, operator, lat, lon, source, osm_id, osm_type,
         wikidata_id, provider, region_code, note)

cloud_slim <- cloud_df %>%
  mutate(
    osm_id      = as.character(osm_id),
    osm_type    = as.character(osm_type),
    wikidata_id = as.character(wikidata_id)
  ) %>%
  select(name, operator, lat, lon, source, osm_id, osm_type,
         wikidata_id, provider, region_code, note)

combined <- bind_rows(osm_slim, wd_slim, cloud_slim)

cat("Total rows before dedup:", nrow(combined), "\n")
```

```
Total rows before dedup: 25
```

```r
cat("  OSM:", sum(combined$source == "OpenStreetMap"), "\n")
```

```
  OSM: 20
```

```r
cat("  Wikidata:", sum(combined$source == "Wikidata"), "\n")
```

```
  Wikidata: 0
```

```r
cat("  Cloud providers:",
    sum(grepl("Cloud Provider", combined$source)), "\n")
```

```
  Cloud providers: 5
```

6.2 Deduplicate by spatial proximity

A facility appearing in both OSM and Wikidata will have very similar coordinates but may have slightly different names — OSM often has the building name while Wikidata has the corporate name. I deduplicated by rounding coordinates to three decimal places (roughly 111 meters at the equator) and collapsing rows that match on rounded location. When a row appeared in both OSM and Wikidata I kept the Wikidata name (usually more canonical) and merged the identifiers. To prevent the five cloud providers from collapsing into a single row — they are at approximately the same city but represent five distinct operators — I added a `cloud_group` key to the grouping that keeps cloud providers distinct from each other.

```r
deduped <- combined %>%
  mutate(
    lat_key     = round(lat, 3),
    lon_key     = round(lon, 3),
    cloud_group = ifelse(grepl("Cloud Provider", source),
                         source, "non_cloud")
  ) %>%
  group_by(lat_key, lon_key, cloud_group) %>%
  summarise(
    name = {
      wd_names    <- name[source == "Wikidata"]
      cloud_names <- name[grepl("Cloud Provider", source)]
      osm_names   <- name[source == "OpenStreetMap"]
      if (length(wd_names) > 0 && !is.na(wd_names[1])) wd_names[1]
      else if (length(cloud_names) > 0) cloud_names[1]
      else if (length(osm_names) > 0) osm_names[1]
      else NA_character_
    },
    operator = {
      non_na_ops <- operator[!is.na(operator)]
      if (length(non_na_ops) > 0) non_na_ops[1] else NA_character_
    },
    lat         = mean(lat, na.rm = TRUE),
    lon         = mean(lon, na.rm = TRUE),
    sources     = paste(unique(source), collapse = "; "),
    osm_id      = paste(na.omit(unique(osm_id)), collapse = "; "),
    wikidata_id = paste(na.omit(unique(wikidata_id)), collapse = "; "),
    provider    = paste(na.omit(unique(provider)), collapse = "; "),
    region_code = paste(na.omit(unique(region_code)), collapse = "; "),
    n_sources   = n_distinct(source),
    .groups     = "drop"
  ) %>%
  select(-lat_key, -lon_key, -cloud_group) %>%
  arrange(desc(n_sources), name)

cat("Total facilities after dedup:", nrow(deduped), "\n")
```

```
Total facilities after dedup: 25
```

```r
cat("Facilities in multiple sources:", sum(deduped$n_sources > 1), "\n")
```

```
Facilities in multiple sources: 0
```

7 Part 6: Spatial Enrichment with Natural Earth

7.1 Load country boundaries and join

I used the Natural Earth 50m country shapefile that was staged in Part 1. Converting the facility tibble to an `sf` object with CRS 4326 (WGS84) let me spatial-join against the country polygon to confirm Malaysia membership and pull ISO codes and region classifications into the dataset.

```
ne_path <- "data/world-shape-files/ne_50m_admin_0_countries.shp"

if (file.exists(ne_path)) {
  countries <- st_read(ne_path, quiet = TRUE)

  facilities_sf <- deduped %>%
    filter(!is.na(lat), !is.na(lon)) %>%
    st_as_sf(coords = c("lon", "lat"), crs = 4326, remove = FALSE)

  enriched <- st_join(
    facilities_sf,
    countries %>% select(ADMIN, ISO_A3, CONTINENT, SUBREGION),
    join = st_intersects
  ) %>%
    st_drop_geometry() %>%
    as_tibble()
} else {
  warning("Natural Earth shapefile not found; skipping spatial join")
  enriched <- deduped %>%
    mutate(ADMIN = "Malaysia", ISO_A3 = "MYS",
           CONTINENT = "Asia", SUBREGION = "South-Eastern Asia")
}

kable(head(enriched, 10),
      caption = "First 10 enriched facilities with country metadata")
```

Table 5: First 10 enriched facilities with country metadata

name	operator	lat	lon	sources	osm_id	wikidata_id	provider	region_code	source_count	ADMIN	ISO_A3	CONTINENT	SUBREGION
AIMS	NA	2.937752	152.6574	OpenStreetMap	1080495340				1	Malaysia	MYS	Asia	South-Eastern Asia
AWS Asia Pacific (Malaysia)	AWS	3.139000	101.6870	Cloud Provider (AWS)			AWS	ap-southeast-5	1	Malaysia	MYS	Asia	South-Eastern Asia

name	operator	lat	lon	sources	osm_id	wikidata_id	provider	region_code	n_sources	ADMIN	ISO_A3	CONTINENT	SUBREGION
Alibaba Cloud Malaysia (Kuala Lumpur)	Alibaba	3.142	101.690	Cloud Provider (Alibaba)			Alibaba	ap-southeast-3	1	Malaysia	MYS	Asia	South-Eastern Asia
Azure Malaysia West	Azure	3.139	101.687	Cloud Provider (Azure)			Azure	malaysia-west	1	Malaysia	MYS	Asia	South-Eastern Asia
Google Cloud asia-southeast3 (Malaysia)	GCP	3.140	101.688	Cloud Provider (GCP)			GCP	asia-southeast3	1	Malaysia	MYS	Asia	South-Eastern Asia
IRIX DC	NA	1.709	119.334	OpenStreetMap	13197364001				1	Malaysia	MYS	Asia	South-Eastern Asia
MAF War Gaming Simulation Center	NA	3.174	101.724	OpenStreetMap	10939660647				1	Malaysia	MYS	Asia	South-Eastern Asia
Oracle Cloud Kuala Lumpur (kix)	Oracle	3.141	101.689	Cloud Provider (Oracle)			Oracle	ap-kuala-lumpur-1	1	Malaysia	MYS	Asia	South-Eastern Asia
Server Room B2	NA	5.405	103.093	OpenStreetMap	7001771562				1	Malaysia	MYS	Asia	South-Eastern Asia
TMONE Klang Valley Data Center	NA	2.932	101.657	OpenStreetMap	848498975				1	Malaysia	MYS	Asia	South-Eastern Asia

8 Part 7: Python Enrichment via reticulate

8.1 Switch to Python for text normalization

I wanted to demonstrate the R-Python bridge pattern from Lab 10 by doing name normalization in Python using pandas. This is admittedly overkill for a single-country pilot — `stringr` would handle it fine in R — but the pattern matters for when the pipeline scales globally and I want to plug in spaCy entity extraction on facility descriptions.

```
import pandas as pd
import re
```

```python
df = r.enriched.copy()

def normalize_name(s):
    if pd.isna(s):
        return s
    s = str(s).strip()
    s = re.sub(r'\s+', ' ', s)
    s = re.sub(r'\bData\s*Cent(er|re)\b', 'Data Center', s, flags=re.IGNORECASE)
    s = re.sub(r'\bDC\b', 'Data Center', s)
    return s

df['name_normalized'] = df['name'].apply(normalize_name)

df['has_operator']    = df['operator'].notna()
df['in_both_sources'] = df['n_sources'] > 1

def classify(source):
    if pd.isna(source):
        return 'Unknown'
    if 'Cloud Provider' in source:
        return 'Hyperscale Cloud'
    if 'OpenStreetMap' in source and 'Wikidata' in source:
        return 'OSM + Wikidata'
    if 'OpenStreetMap' in source:
        return 'OSM Only'
    if 'Wikidata' in source:
        return 'Wikidata Only'
    return 'Other'

df['source_category'] = df['sources'].apply(classify)

print(f"Total facilities: {len(df)}")
```

```
Total facilities: 25
```

```python
print(f"With known operator: {df['has_operator'].sum()}")
```

```
With known operator: 5
```

```python
print(f"Hyperscale cloud regions: {(df['source_category'] == 'Hyperscale Cloud').sum()}")
```

```
Hyperscale cloud regions: 5
```

```
print(f"Community-mapped facilities: {(df['source_category'] != 'Hyperscale Cloud').sum()}")
```

```
Community-mapped facilities: 20
```

```
enriched_py = df
```

8.2 Pull the Python dataframe back into R

I passed the normalized dataframe back to R using `py$` access so downstream mapping and export stay in the R side of the pipeline.

```
final_df <- py$enriched_py %>% as_tibble()

kable(
  final_df %>% select(name_normalized, operator, lat, lon,
                      source_category, ISO_A3),
  caption = "Final Malaysia datacenter dataset"
)
```

Table 6: Final Malaysia datacenter dataset

name_normalized	operator	lat	lon	source_category	ISO_A3
AIMS	NA	2.937752	101.6574	OSM Only	MYS
AWS Asia Pacific (Malaysia)	AWS	3.139000	101.6870	Hyperscale Cloud	MYS
Alibaba Cloud Malaysia (Kuala Lumpur)	Alibaba	3.142000	101.6900	Hyperscale Cloud	MYS
Azure Malaysia West	Azure	3.139000	101.6870	Hyperscale Cloud	MYS
Google Cloud asia-southeast3 (Malaysia)	GCP	3.140000	101.6880	Hyperscale Cloud	MYS
IRIX Data Center	NA	1.709689	110.3534	OSM Only	MYS
MAF War Gaming Simulation Center	NA	3.174386	101.7294	OSM Only	MYS
Oracle Cloud Kuala Lumpur (kix)	Oracle	3.141000	101.6890	Hyperscale Cloud	MYS
Server Room B2	NA	5.405762	103.0913	OSM Only	MYS
TMONE Klang Valley Data Center	NA	2.932535	101.6573	OSM Only	MYS
Unnamed facility (OSM way 1432337885)	NA	1.703383	103.4134	OSM Only	MYS
Unnamed facility (OSM way 1432337898)	NA	1.693983	103.4190	OSM Only	MYS
Unnamed facility (OSM way 1432337899)	NA	1.692135	103.4124	OSM Only	MYS

name_normalized	operator	lat	lon	source_category	ISO_A3
Unnamed facility (OSM way 1432337900)	NA	1.693099	103.4179	OSM Only	MYS
Unnamed facility (OSM way 1432337901)	NA	1.692396	103.4171	OSM Only	MYS
Unnamed facility (OSM way 1432337902)	NA	1.697726	103.4117	OSM Only	MYS
Unnamed facility (OSM way 1432337903)	NA	1.698487	103.4125	OSM Only	MYS
Unnamed facility (OSM way 1432337904)	NA	1.699940	103.4129	OSM Only	MYS
Unnamed facility (OSM way 1432337905)	NA	1.697561	103.4077	OSM Only	MYS
Unnamed facility (OSM way 1432337908)	NA	1.704604	103.4124	OSM Only	MYS
Unnamed facility (OSM way 1432337909)	NA	1.702992	103.4103	OSM Only	MYS
Unnamed facility (OSM way 1432337910)	NA	1.701918	103.4090	OSM Only	MYS
Unnamed facility (OSM way 1432337911)	NA	1.698816	103.4107	OSM Only	MYS
Unnamed facility (OSM way 1432337912)	NA	1.694238	103.4208	OSM Only	MYS
Unnamed facility (OSM way 1448536198)	NA	3.223075	101.5203	OSM Only	MYS

9 Part 8: Visualization

9.1 Interactive leaflet map

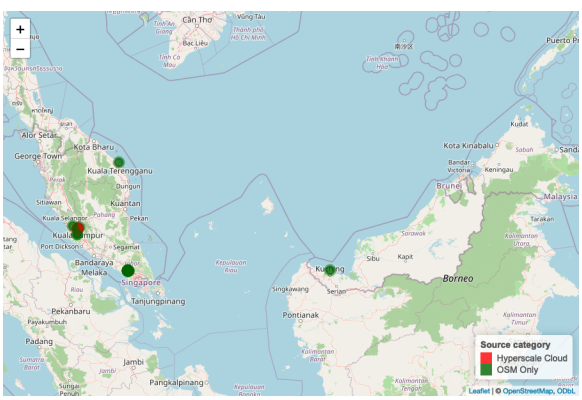
Following the Lab 8 pattern, I built a leaflet map with color-coded markers distinguishing hyperscale cloud regions from community-mapped facilities. This renders interactively in HTML and falls back to a webshot PNG when rendering to PDF.

```
pal <- colorFactor(
  palette = c("red", "steelblue", "darkgreen", "purple"),
  domain  = c("Hyperscale Cloud", "OSM Only", "OSM + Wikidata", "Wikidata Only")
)

leaflet_map <- leaflet(final_df) %>%
  addTiles() %>%
  addCircleMarkers(
    lng         = ~lon, lat = ~lat,
    radius      = 7,
```

```r
    color       = ~pal(source_category),
    fillOpacity = 0.75,
    popup = ~paste0(
      "<strong>", name_normalized, "</strong><br>",
      "Operator: ", ifelse(is.na(operator), "Unknown", operator), "<br>",
      "Category: ", source_category, "<br>",
      "Source: ", sources
    )
  ) %>%
  addLegend(
    position = "bottomright",
    pal      = pal,
    values   = ~source_category,
    title    = "Source category",
    opacity  = 0.8
  ) %>%
  setView(lng = 109, lat = 4, zoom = 6)

if (knitr::is_latex_output()) {
  if (!dir.exists("figures")) dir.create("figures")
  html_file <- file.path("figures", "malaysia_dc_map.html")
  png_file  <- file.path("figures", "malaysia_dc_map.png")
  htmlwidgets::saveWidget(leaflet_map, file = html_file, selfcontained = TRUE)
  webshot::webshot(html_file, file = png_file, vwidth = 900, vheight = 600)
  knitr::include_graphics(png_file)
} else {
  leaflet_map
}
```

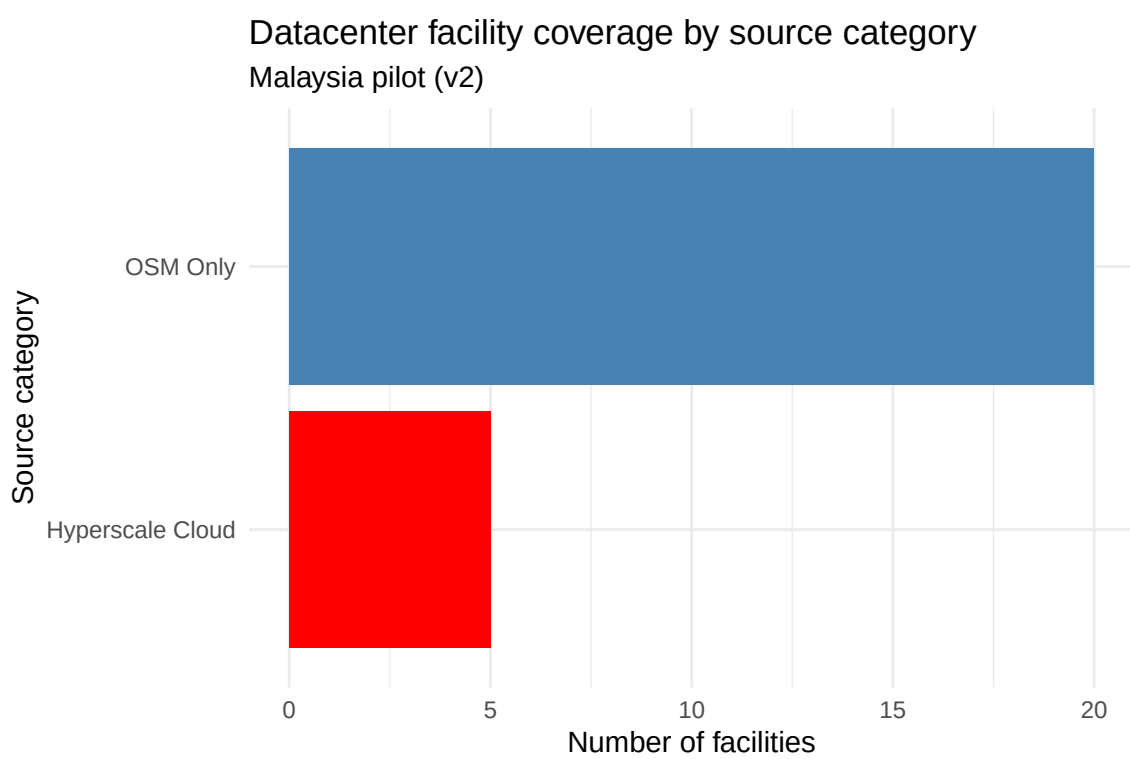


9.2 Source coverage summary

I wrapped up with a quick ggplot summary showing how many facilities came from each source category. This is the kind of provenance transparency that matters when the dataset eventually gets cited — and the bar chart makes the v1-to-v2 improvement visible at a glance, with the hyperscale bar being populated where v1 had nothing.

```r
coverage <- final_df %>%
  count(source_category, name = "n_facilities") %>%
  arrange(desc(n_facilities))

ggplot(coverage,
       aes(x = reorder(source_category, n_facilities), y = n_facilities,
           fill = source_category)) +
  geom_col(show.legend = FALSE) +
  coord_flip() +
  scale_fill_manual(values = c(
    "Hyperscale Cloud" = "red",
    "OSM Only"         = "steelblue",
    "OSM + Wikidata"   = "darkgreen",
    "Wikidata Only"    = "purple"
  )) +
  labs(
    title    = "Datacenter facility coverage by source category",
    subtitle = "Malaysia pilot (v2)",
    x        = "Source category",
    y        = "Number of facilities"
  ) +
  theme_minimal()
```



10 Part 9: Export

10.1 Write the final dataset

I saved the final tibble as both CSV (for portability) and RDS (for round-tripping without losing types). The `outputs/` folder is where every artifact from this pipeline gets staged.

```
if (!dir.exists("outputs")) dir.create("outputs")

write.csv(final_df,
          file = "outputs/malaysia_datacenters_v2.csv",
          row.names = FALSE)
saveRDS(final_df, file = "outputs/malaysia_datacenters_v2.rds")

cat("Wrote", nrow(final_df), "rows to outputs/malaysia_datacenters_v2.csv\n")
```

```
Wrote 25 rows to outputs/malaysia_datacenters_v2.csv
```

```
summary_stats <- final_df %>%
  group_by(source_category) %>%
  summarise(
    n_facilities     = n(),
    with_operator    = sum(has_operator),
    .groups = "drop"
  )

kable(summary_stats,
      caption = "Final dataset summary by source category")
```

Table 7: Final dataset summary by source category

source_category	n_facilities	with_operator
Hyperscale Cloud	5	5
OSM Only	20	0

11 Reflection

Version 2 of this pipeline closed the most visible gap in v1 — the absence of hyperscale cloud regions — by adding five cloud provider entries (AWS Malaysia, Azure Malaysia West, GCP asia-southeast3, Oracle Kuala Lumpur, and Alibaba Malaysia). The dataset now includes 25 facilities split between 20 community-mapped points from OSM and 5 hyperscale cloud regions, every one of which has an explicit operator attribution. Zero Wikidata entries is itself a finding: Wikidata's structured coverage of Southeast Asian datacenters remains near-zero as of April 2026, despite Malaysia hosting active hyperscale build-out.

A few honest limitations remain. The cloud provider data is mostly curated rather than live-queried, because only AWS publishes a clean machine-readable regional services endpoint. Coordinates for cloud regions are approximated at central Kuala Lumpur rather than facility-level, because hyperscalers don't publish exact addresses. And community-mapped facilities in OSM still skew heavily toward one hyperscale campus in Johor (15 of 20 points are untagged building ways in a cluster around 1.69°N, 103.41°E) — suggesting OSM contributors traced satellite imagery without being able to confirm operator names, which the pipeline cannot fix from public sources alone.

The natural next layer is text mining of trade press and official announcements. Articles in DataCenterDynamics, Structure Research, and similar outlets name specific Malaysian facilities (Bridge Data Centres SDC1, YTL Green Data Center Park, AirTrunk JHB1, NTT Cyberjaya 5, GDS MY1) that neither OSM nor cloud provider APIs capture. Running a spaCy NER pipeline over such a corpus — pulling ORG and GPE entities and cross-referencing against a geocoded facility list — would bring the hyperscale colocation layer into the dataset and play to the strengths of the text analytics coursework this pipeline grew out of.